



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



Estructura de datos con memoria dinámica

Las estructuras de datos con memoria dinámica se usan para **optimizar** el uso de **memoria** cuando **almacenamos** datos

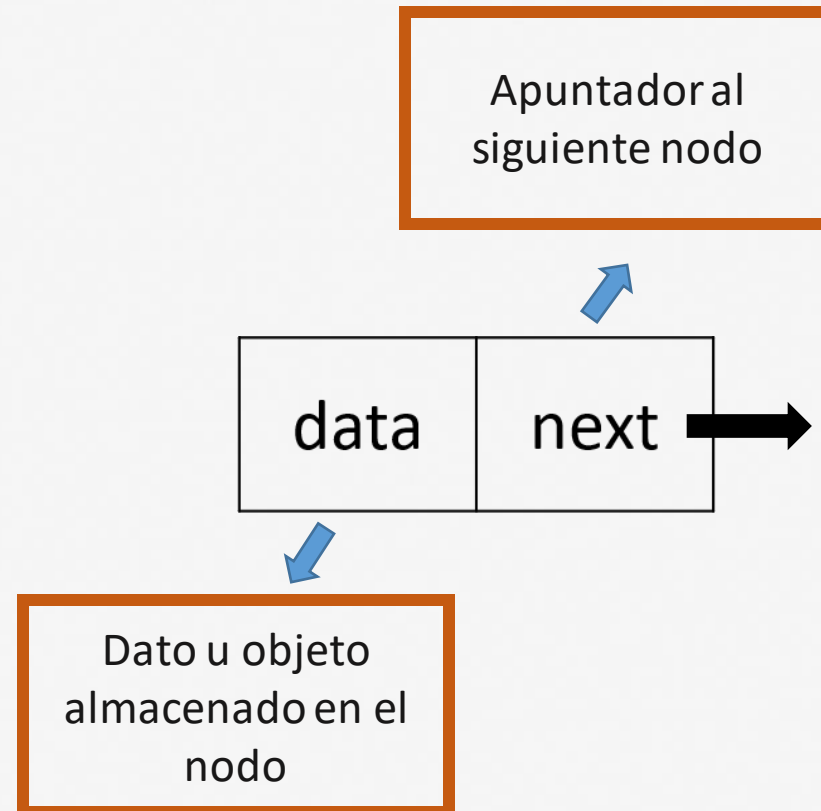
Estas estructuras siguen siendo **colecciones** de datos, pero con formas de acceso **diferentes**

Las estructuras de datos de memoria dinámica son:

- *Listas enlazadas simples*
- *Listas enlazadas dobles*
- *Listas circulares*
- Pilas
- Colas
- Árboles

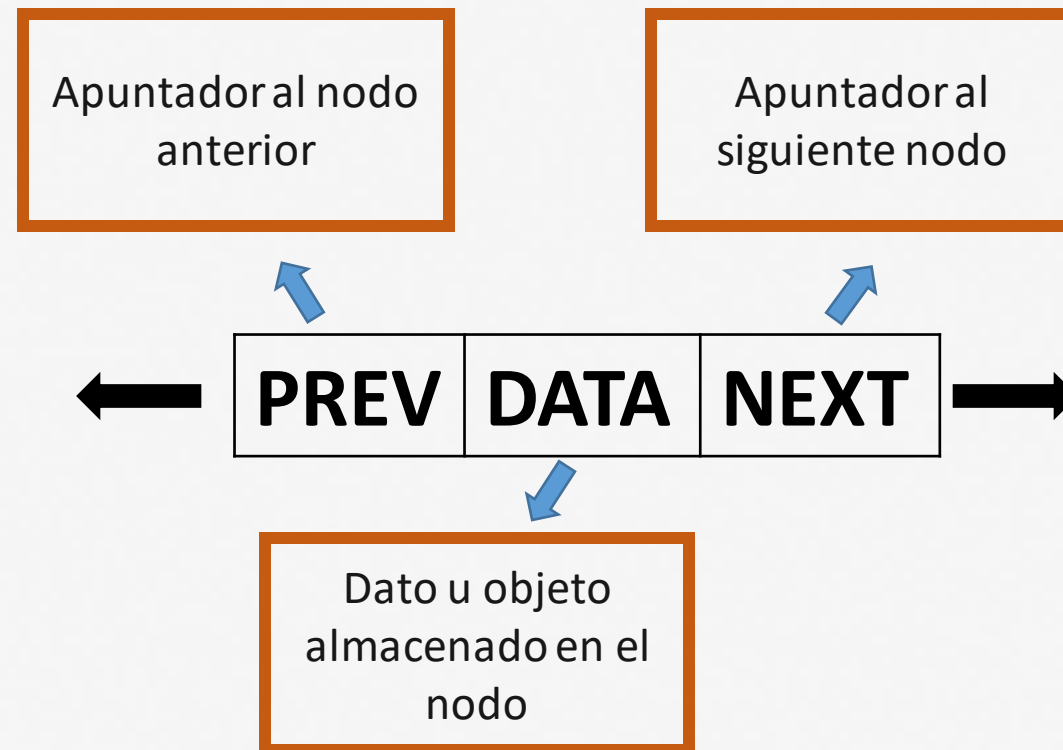
QUÉ ES UN NODO - recordemos

Es un elemento que almacena: un dato y un enlace hacia otro nodo



LISTAS DOBLEMENTE ENLAZADAS

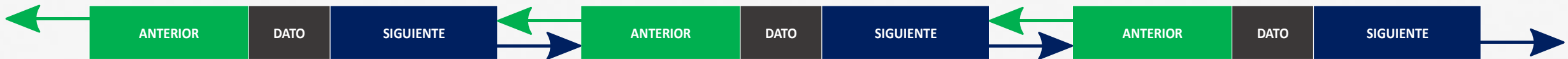
Una lista doblemente enlazada es una lista lineal en la que cada nodo tiene dos enlaces, uno al nodo siguiente, y otro al anterior.



LISTAS DOBLEMENTE ENLAZADAS

A diferencia de una lista enlazada, la lista doblemente enlazada se puede navegar y ser analizada en ambas direcciones.

La referencia al siguiente nodo ayuda a que nos podamos mover hacia adelante y la referencia al nodo anterior nos ayuda a movernos en dirección contraria.



LISTAS DOBLEMENTE ENLAZADAS

Definición de un nodo dentro de una lista doblemente enlazada

```
class Nodo:  
    def __init__(self, elemento):  
        self.elemento = elemento  
        self.siguiente = None  
        self.anterior = None
```

LISTAS DOBLEMENTE ENLAZADAS

Definición de la clase de la lista doblemente enlazada

```
class doubleList:  
    def __init__(self):  
        self.head = None
```

Recordemos que el atributo head indica el primer elemento almacenado en la lista, ese atributo se inicializa en None, que indica que la lista está vacía

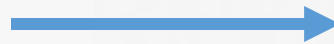
LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento en el head de la lista vacía

```
class doubleList:
    def __init__(self):
        self.head = None

    def insertar_lista_vacia(self, dato):

        if self.head is None:
            nuevoNodo = Nodo(dato)
            self.head = nuevoNodo
        else:
            print("La lista no está vacía")
```

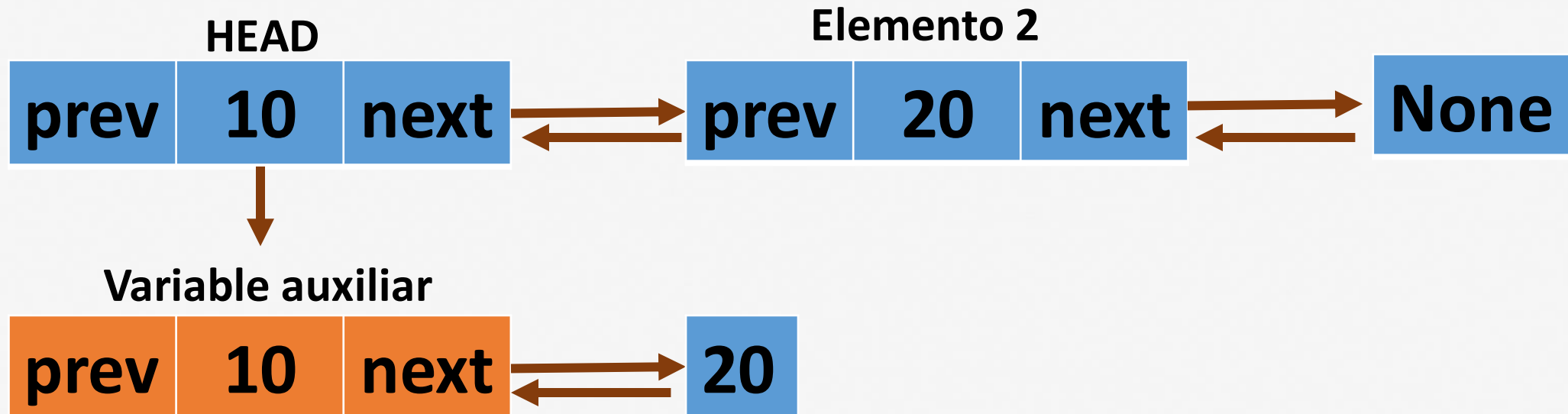


Primero preguntamos si la lista se encuentra vacía, en caso afirmativo creamos el nodo con el dato ingresado y lo ubicamos en el head de nuestra lista

LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos en la lista enlazada

Elemento nuevo



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos en la lista enlazada

Elemento nuevo



HEAD



Elemento 2

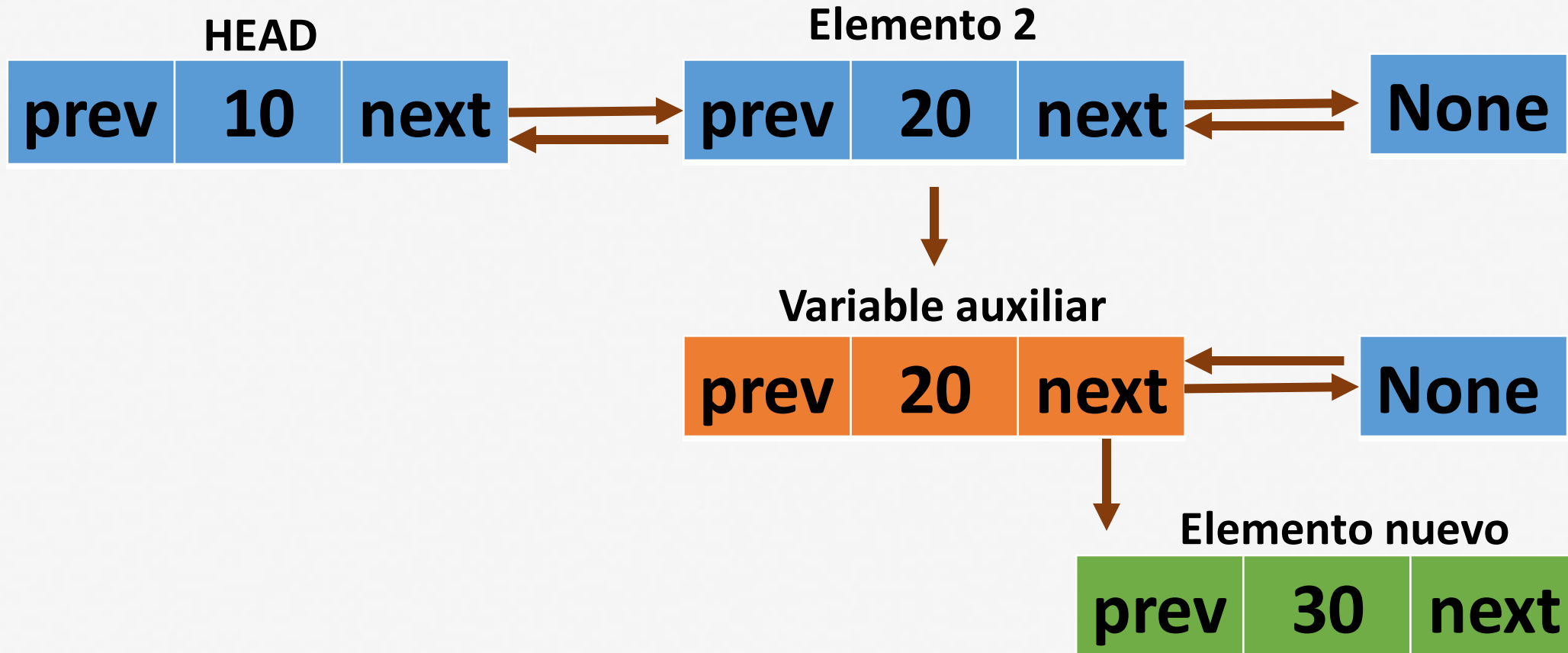


Variable auxiliar



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos en la lista enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos en la lista enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos en la lista enlazada

```
def insertar_siguiente(self, dato):
```

```
    if self.head is None:
```

```
        self.insertar_lista_vacia(dato)
```

```
        return
```

```
    else:
```

```
        # si la lista no está vacía buscamos el elemento final:
```

```
        apuntador = self.head
```

```
        while apuntador.siguiente is not None:
```

```
            apuntador = apuntador.siguiente
```

```
        nuevoNodo = Nodo(dato)
```

```
        apuntador.siguiente = nuevoNodo
```

```
        nuevoNodo.anterior = apuntador
```

Si la lista está vacía añadimos el nuevo elemento en el head

Recorremos toda la lista hasta encontrar el elemento final

Creamos el nuevo nodo
El nuevo nodo lo añadimos al atributo siguiente del último nodo de la lista

LISTAS DOBLEMENTE ENLAZADAS

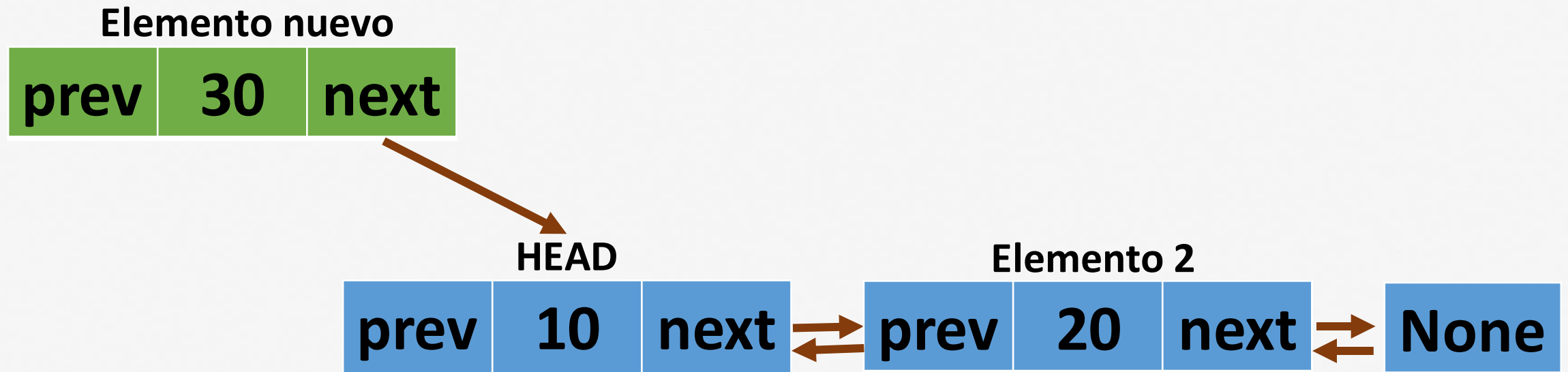
Función para insertar elementos en la lista enlazada

```
def insertar_siguiente(self, dato):  
  
    if self.head is None:  
        self.insertar_lista_vacia(dato)  
        return  
  
    else:  
        # si la lista no está vacía buscamos el elemento final:  
        apuntador = self.head  
  
        while apuntador.siguiente is not None:  
            apuntador = apuntador.siguiente  
  
        nuevoNodo = Nodo(dato)  
        apuntador.siguiente = nuevoNodo  
        nuevoNodo.anterior = apuntador
```

```
lista = doubleList()  
lista.insertar_siguiente(3)  
print(lista.head.elemento)
```

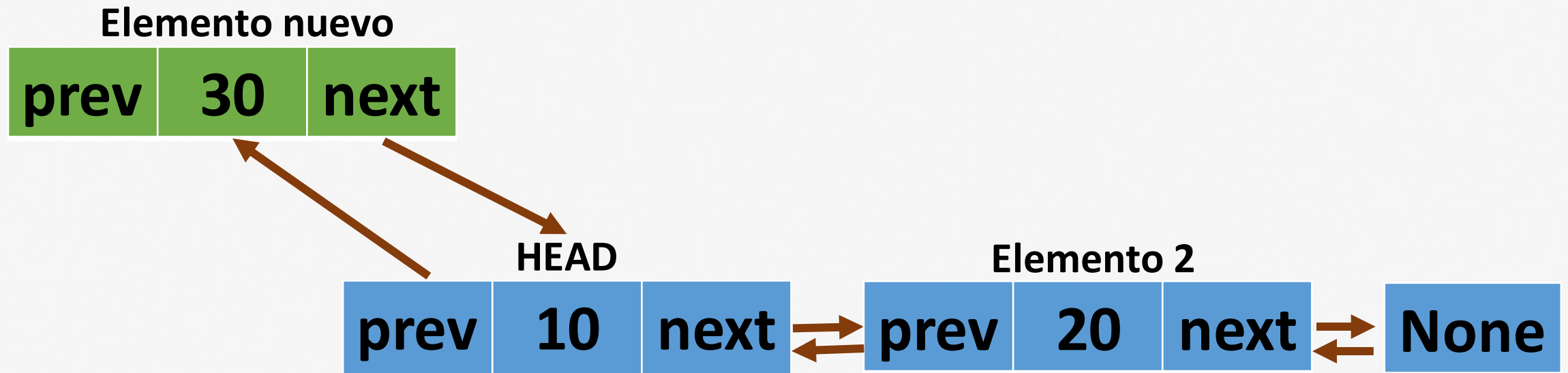

LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos al inicio de la lista enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos al inicio de la lista enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos al inicio de la lista enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar elementos al inicio de la lista enlazada

```
def insertar_inicio(self, dato):  
  
    if self.head is None:  
        self.insertar_lista_vacia(dato)  
        return  
  
    nuevoNodo = Nodo(dato)  
  
    nuevoNodo.siguiente = self.head  
    self.head.anterior = nuevoNodo  
    self.head = nuevoNodo
```

Si la lista está vacía añadimos el nuevo elemento en el head

Creamos el nuevo nodo

El nuevo nodo ahora será el nuevo head
El head anterior se desplaza

LISTAS DOBLEMENTE ENLAZADAS

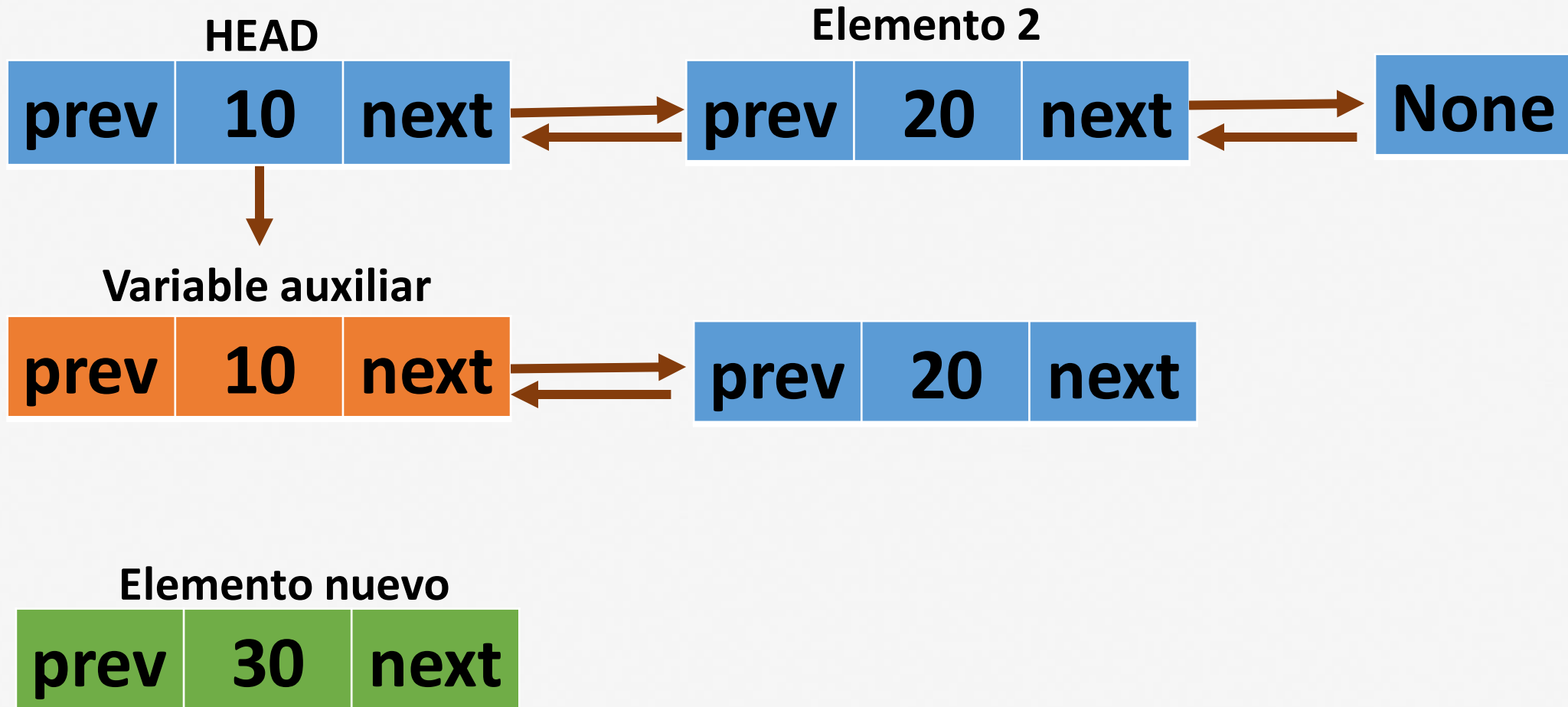
Función para insertar elementos al inicio de la lista enlazada

```
def insertar_inicio(self, dato):  
  
    if self.head is None:  
        self.insertar_lista_vacia(dato)  
        return  
  
    nuevoNodo = Nodo(dato)  
  
    nuevoNodo.siguiente = self.head  
    self.head.anterior = nuevoNodo  
    self.head = nuevoNodo
```

```
lista.insertar_inicio(5)  
print(lista.head.elemento)  
print(lista.head.siguiente.elemento)
```

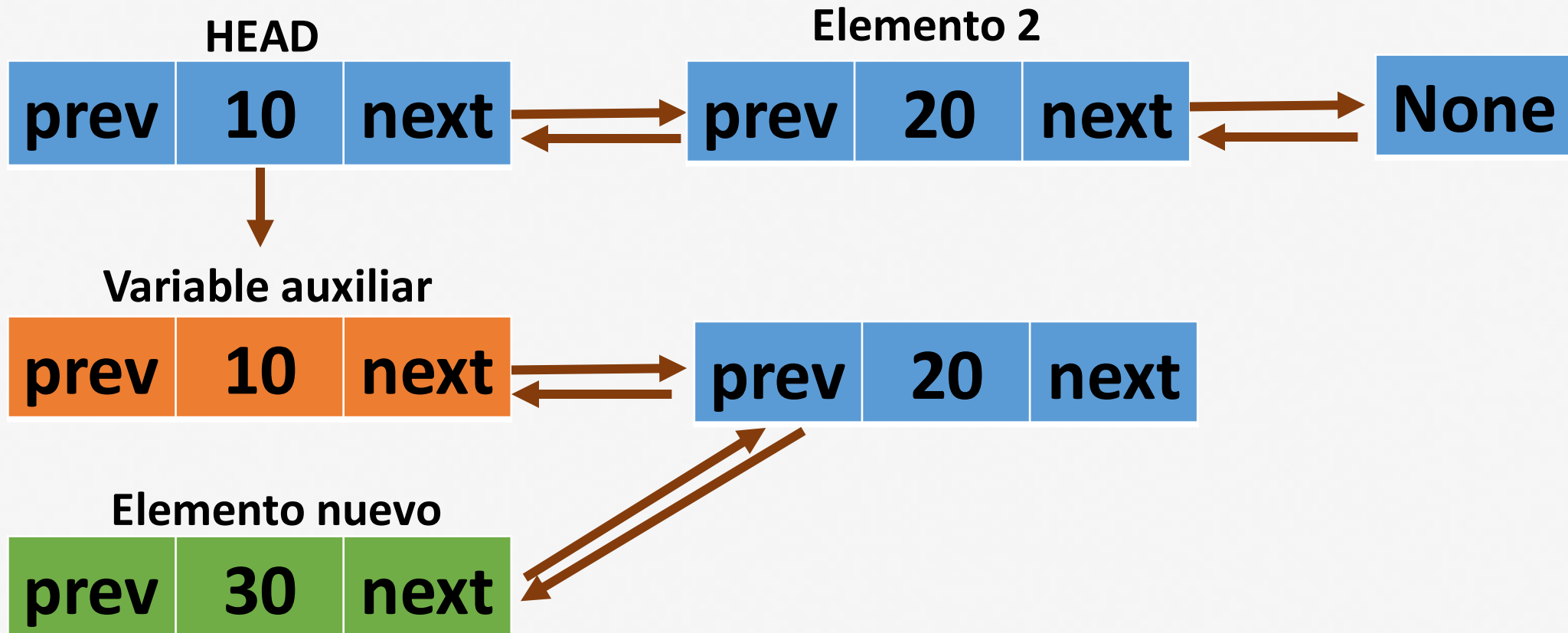
LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista



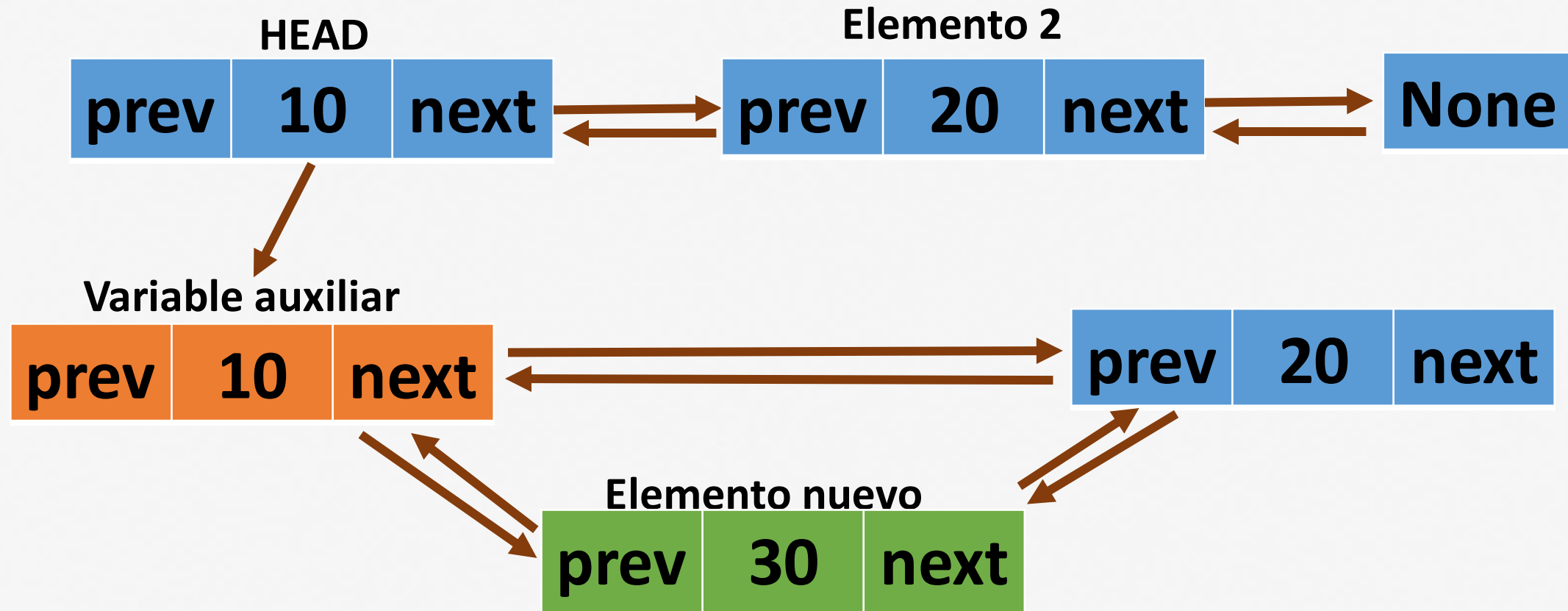
LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista



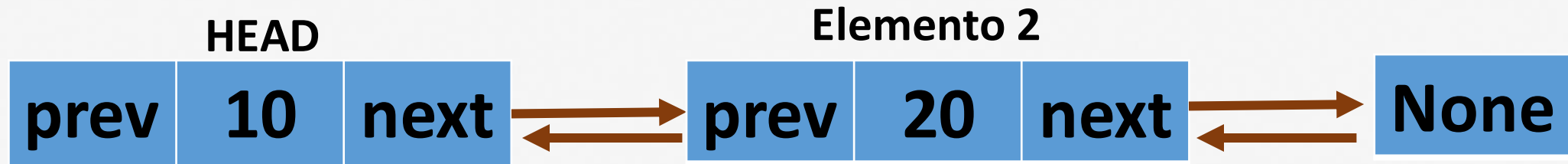
LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista



LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista

```
def insertar_despues(self, valor, nuevo_dato):  
    nuevoNodo = Nodo(nuevo_dato)  
    apuntador = self.head  
    while apuntador:  
        if apuntador.elemento == valor:  
            nuevoNodo.siguiente = apuntador.siguiente  
            if apuntador.siguiente:  
                apuntador.siguiente.anterior = nuevoNodo  
            apuntador.siguiente = nuevoNodo  
            nuevoNodo.anterior = apuntador  
            return  
        apuntador = apuntador.siguiente
```

→ Creamos el nuevo nodo

→ Recorremos todos los elementos de la lista hasta encontrar el elemento deseado

→ Hacemos de nuevo las conexiones

LISTAS DOBLEMENTE ENLAZADAS

Función para insertar un elemento después de un valor dentro de la lista

```
def insertar_despues(self, valor, nuevo_dato):  
    nuevoNode = Node(nuevo_dato)  
    apuntador = self.head  
    while apuntador:  
        if apuntador.elemento == valor:  
            nuevoNode.siguiente = apuntador.siguiente  
            if apuntador.siguiente:  
                apuntador.siguiente.anterior = nuevoNode  
            apuntador.siguiente = nuevoNode  
            nuevoNode.anterior = apuntador  
            return  
    apuntador = apuntador.siguiente
```

```
lista.insertar_despues(5,2)  
print(lista.head.elemento)  
print(lista.head.siguiente.elemento)  
print(lista.head.siguiente.siguiente.elemento)
```

Cree el método `insertar_final` que, dado un elemento dentro de la lista, y un nuevo dato, inserte el elemento en la posición final

Cree la lista: 3, 7, 9

Llame el método `insertar_final(5)`

El método deberá agregar el número 5 una posición final para obtener:

3,7,9,5

```
def insertar_antes(self, valor, nuevo_dato):  
    nuevoNodo = Nodo(nuevo_dato)  
    apuntador = self.head  
    while apuntador:  
        if apuntador.elemento == valor:  
            nuevoNodo.anterior = apuntador.anterior  
            if apuntador.anterior:  
                apuntador.anterior.siguiente = nuevoNodo  
            else:  
                self.head = nuevoNodo  
            nuevoNodo.siguiente = apuntador  
            apuntador.anterior = nuevoNodo  
            return  
    apuntador = apuntador.siguiente
```

```
lista1 = doubleList()  
lista1.insertar_siguiente(3)  
lista1.insertar_siguiente(7)  
lista1.insertar_siguiente(9)  
lista1.insertar_antes(7,5)  
print(lista1.head.elemento)  
print(lista1.head.siguiente.elemento)  
print(lista1.head.siguiente.siguiente.elemento)  
print(lista1.head.siguiente.siguiente.siguiente.elemento)
```

LISTAS DOBLEMENTE ENLAZADAS

Función para imprimir los elementos dentro de la lista doblemente enlazada

```
def imprimir(self):  
    apuntador = self.head  
    while apuntador:  
        print(apuntador.elemento, end=" <-> ")  
        apuntador = apuntador.siguiente  
    print("None")
```

```
lista1.imprimir()
```




Institución
Universitaria
Reacreditada en Alta Calidad

ACTIVIDAD

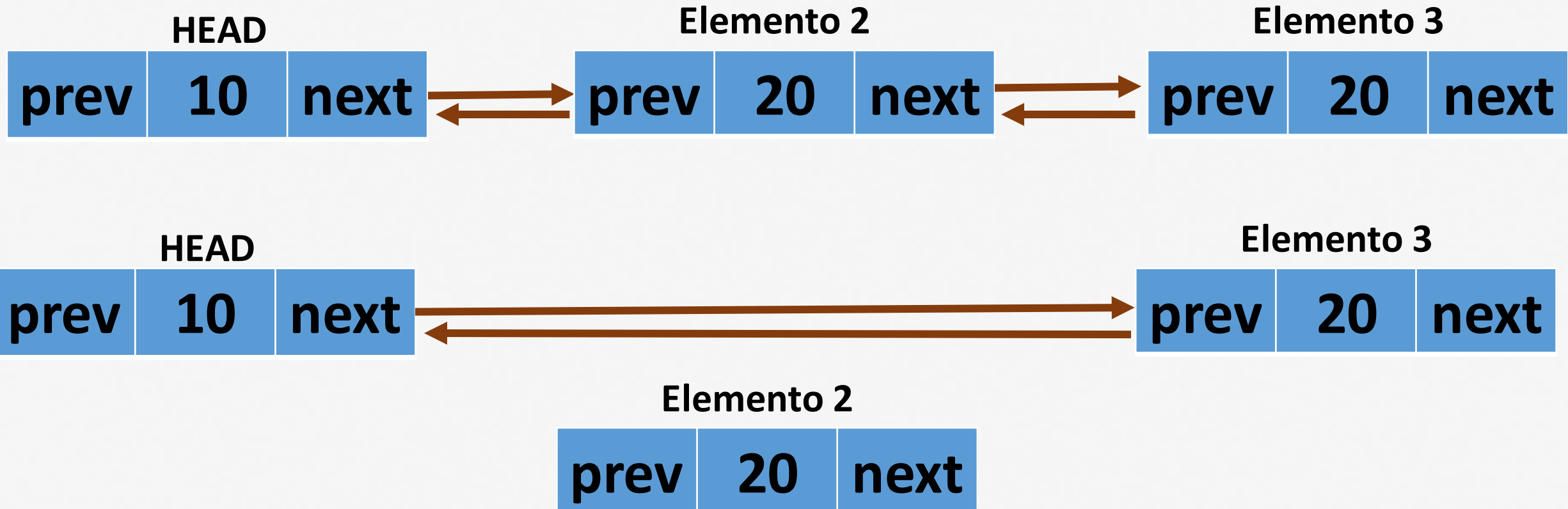
Cree el método `imprimir_reverso` que imprima los elementos de la lista doblemente enlazada de atrás hacia adelante

```
def imprimir_reverso(self):  
    apuntador = None  
    if self.head:  
        apuntador = self.head  
        while apuntador.siguiente:  
            apuntador = apuntador.siguiente  
  
    while apuntador:  
        print(apuntador.elemento, end=" <-> ")  
        apuntador = apuntador.anterior
```

```
lista1.imprimir_reverso()
```

LISTAS DOBLEMENTE ENLAZADAS

Función para eliminar un elemento dentro de una lista doblemente enlazada



LISTAS DOBLEMENTE ENLAZADAS

Función para eliminar un elemento dentro de una lista doblemente enlazada

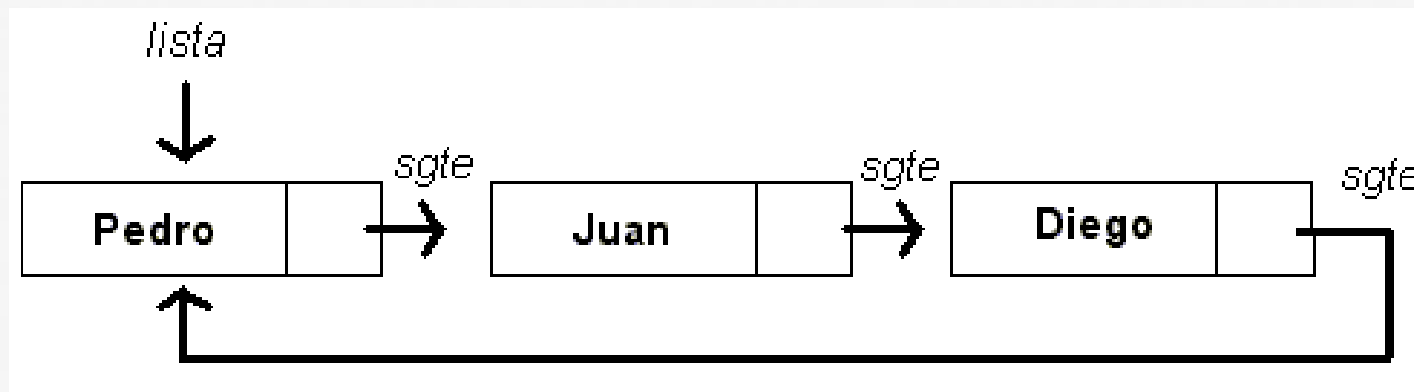
```
def eliminar(self, elemento):  
    apuntador = self.head  
    while apuntador:  
        if apuntador.elemento == elemento:  
            if apuntador.anterior:  
                apuntador.anterior.siguiente = apuntador.siguiente  
            if apuntador.siguiente:  
                apuntador.siguiente.anterior = apuntador.anterior  
            if apuntador == self.head:  
                self.head = apuntador.siguiente  
            return  
        apuntador = apuntador.siguiente
```

Si el elemento a eliminar tiene nodos conectados atrás y adelante, lo que hacemos es conectar esos nodos entre sí, eliminando la conexión que existe con el nodo a eliminar

LISTAS CIRCULARES - ENLAZADAS

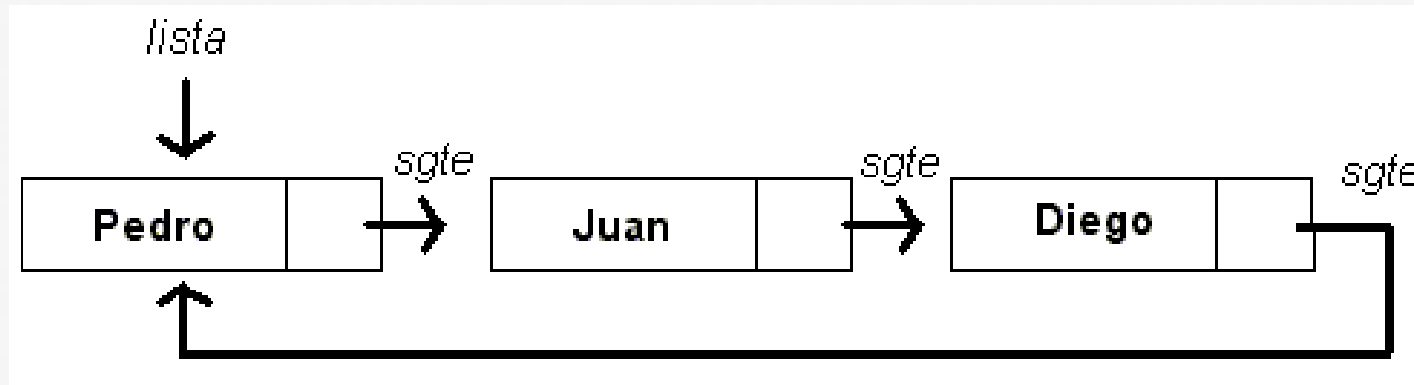
La característica distintiva de una lista simplemente enlazada circular es que el último nodo de la lista apunta de nuevo al primer nodo, creando un ciclo cerrado.

Cada vez que se recorren los elementos de una lista circular, se retorna nuevamente al primer elemento, generando un bucle.



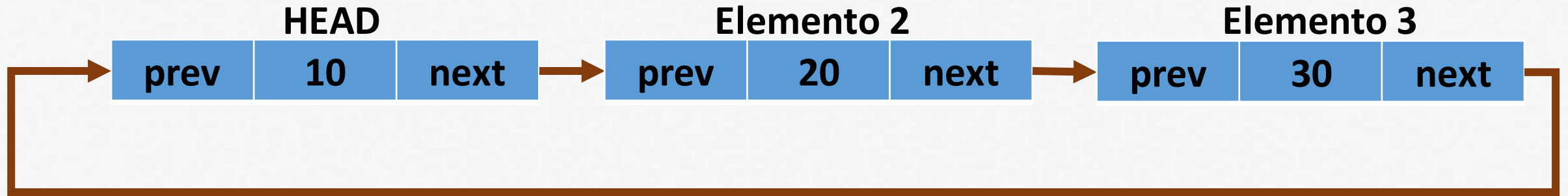
LISTAS CIRCULARES - ENLAZADAS

Cada vez que necesitemos recorrer los elementos de una lista circular, la condición de parada se dará cuando lleguemos al head, de lo contrario estaremos generando un bucle infinito

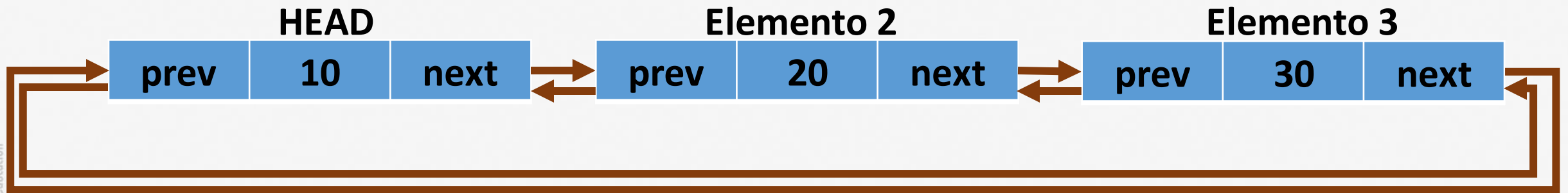


LISTAS CIRCULARES

Pueden ser simplemente enlazadas



Pueden ser doblemente enlazadas



LISTAS CIRCULARES ENLAZADAS SIMPLES

El nodo se define igual que en una lista enlazada simple

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```


LISTAS CIRCULARES ENLAZADAS SIMPLES

El método append para insertar un nuevo elemento dentro de la lista circular

```
def append(self, data):  
    new_node = Node(data)  
    if not self.head:  
        self.head = new_node  
        self.head.next = self.head  
    else:  
        temp = self.head  
        while temp.next != self.head:  
            temp = temp.next  
        temp.next = new_node  
        new_node.next = self.head
```

Si la lista está vacía ubicamos el nuevo elemento en el head

Conectamos el atributo next al head para cerrar el ciclo de la lista circular

Si la lista no está vacía, recorremos todos sus elementos hasta llegar al último elemento antes del head

Conectamos el último elemento al head para cerrar el ciclo

LISTAS CIRCULARES ENLAZADAS SIMPLES

El método append para insertar un nuevo elemento dentro de la lista circular

```
def append(self, data):  
    new_node = Node(data)  
    if not self.head:  
        self.head = new_node  
        self.head.next = self.head  
    else:  
        temp = self.head  
        while temp.next != self.head:  
            temp = temp.next  
        temp.next = new_node  
        new_node.next = self.head
```

```
c11 = CircularLinkedList()  
c11.append(1)  
c11.append(2)  
c11.append(3)
```

LISTAS CIRCULARES ENLAZADAS SIMPLES

El método display para imprimir todos los elementos dentro de la lista circular

```
def display(self):  
    if not self.head:  
        return  
    temp = self.head  
    while True:  
        print(temp.data, end=' -> ')  
        temp = temp.next  
        if temp == self.head:  
            break
```

Si la lista está vacía no imprimimos nada

Comenzamos a imprimir los elementos desde el head

Recorremos la lista imprimiendo sus elementos hasta encontrarnos nuevamente con el head

LISTAS CIRCULARES ENLAZADAS SIMPLES

El método display para imprimir todos los elementos dentro de la lista circular

```
def display(self):  
    if not self.head:  
        return  
    temp = self.head  
    while True:  
        print(temp.data, end=' -> ')  
        temp = temp.next  
        if temp == self.head:  
            break
```

c11.display()

LISTAS CIRCULARES ENLAZADAS SIMPLES

El método count para contar la cantidad de elementos dentro de la lista circular

```
def count(self):  
    count = 0  
    temp = self.head  
    if not temp:  
        return count  
    count = 1  
    temp = temp.next  
    while temp != self.head:  
        count += 1  
        temp = temp.next  
    return count
```

→ Iniciamos el contador en cero

→ Sumamos un 1 tras almacenar el head

→ Recorremos la lista aumentando el contador hasta encontrarnos nuevamente con el head

LISTAS CIRCULARES ENLAZADAS SIMPLES

El método count para contar la cantidad de elementos dentro de la lista circular

```
def count(self):  
    count = 0  
    temp = self.head  
    if not temp:  
        return count  
    count = 1  
    temp = temp.next  
    while temp != self.head:  
        count += 1  
        temp = temp.next  
    return count
```

```
element_count = cll.count()  
print(f"Número de elementos en la lista: {element_count}")
```




Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white swoosh that curves around the end of the lines.